清华 GitLab Oauth 2.0 调用说明

1. 概述

本文档旨在说明如何在现有 Web 应用中集成 GitLab OAuth 2.0 认证服务。集成后，用户可以通过 GitLab 账号安全登录我们的系统。

本集成采用 **Authorization Code Flow with PKCE**（增强安全性的授权码模式），这是目前推荐的安全标准。

2. 集成流程详解

第一步：构建授权跳转链接 (前端)

当用户点击“使用 GitLab 登录”时，前端需引导用户跳转至以下 URL。

Base URL: `https://git.tsinghua.edu.cn/oauth/authorize`

必须包含的参数：

参数名	值/说明
`client_id`	GitLab 生成的 Application ID
`redirect_uri`	必须与 GitLab 后台配置的地址完全一致 (需 URL Encode)
`response_type`	固定值: `code`
`state`	**推荐**: 随机生成的字符串 (用于防止 CSRF 攻击)
`scope`	请求的权限范围，例如: `read_user`
`code_challenge`	**PKCE 必须**: 生成的挑战码 (见下文算法)
`code_challenge_method`	固定值: `S256`

PKCE 算法说明:

1. **生成 `code_verifier`**: 一个 43-128 位的随机字符串（包含字母、数字、`-`、`.`、`_`、`~`）。**请暂存此值，第三步需使用。**
2. **生成 `code_challenge`**: $Base64Url(SHA256(code_verifier))$

示例 URL:

HTTP

```
https://git.tsinghua.edu.cn/oauth/authorize?
client_id=APP_ID&redirect_uri=https%3A%2F%2Fmyapp.com%2Fcallback&response_ty
pe=code&state=xyz123&scope=read_user&code_challenge=YOUR_CHALLENGE_STRING&co
de_challenge_method=S256
```

第二步：接收回调 (后端)

用户同意授权后，GitLab 会将浏览器重定向回配置的 `redirect_uri`，并附带 `code` 和 `state` 参数。

请求示例:

HTTP

```
GET https://myapp.com/callback?code=RETURNED_AUTH_CODE&state=xyz123
```

处理逻辑:

1. **验证 `state`**：检查返回的 `state` 是否与第一步中生成的 `state` 一致。如果不一致，应拒绝请求（防止 CSRF）。
2. **获取 `code`**：提取 `code` 参数用于下一步。

第三步：换取 Access Token (后端)

使用后端服务器向 GitLab 发起 POST 请求，用 `code` 换取用户的 `access_token`。

接口地址: `POST https://git.tsinghua.edu.cn/oauth/token`

Header: `Content-Type: application/json`

Body 参数:

JSON

```
{
  "client_id": "YOUR_APP_ID",
  "client_secret": "YOUR_APP_SECRET",
  "code": "上一步获取的 code",
  "grant_type": "authorization_code",
  "redirect_uri": "与第一步完全一致的 redirect_uri",
```

```
  "code_verifier": "第一步生成的原始 code_verifier 字符串"
}
```

> **注意**: `code_verifier` 是 PKCE 流程验证的关键，如果缺失或不匹配，GitLab 将拒绝颁发令牌。

成功响应:

JSON

```
{
  "access_token": "de6780...",
  "token_type": "bearer",
  "expires_in": 7200,
  "refresh_token": "8257e6...",
  "created_at": 1604791234
}
```

第四步：获取用户信息 (后端)

获取 Token 后，即可调用 GitLab API 获取当前登录用户的身份信息。

接口地址: `GET https://git.tsinghua.edu.cn/api/v4/user`

Header: `Authorization: Bearer <access_token>`

响应示例:

JSON

```
{
  "email": "zhao-yj23@mails.tsinghua.edu.cn",
  "full_data": {
    "avatar_url":
"https://secure.gravatar.com/avatar/af677d1aa1164bf9b7f0e6c34b20c538d801c434
cb616104cd9daffe60ea126e?s=80&d=identicon",
    "bio": "",
    "bot": false,
    "can_create_group": true,
    "can_create_project": true,
    "color_scheme_id": 1,
    "commit_email": "zhao-yj23@mails.tsinghua.edu.cn",
    "confirmed_at": "2023-08-24T22:25:13.037+08:00",
```

```
    "created_at": "2023-08-24T22:25:13.093+08:00",
    "current_sign_in_at": "2025-12-11T11:31:38.152+08:00",
    "discord": "",
    "email": "zhao-yj23@mails.tsinghua.edu.cn",
    "external": false,
    "github": "",
    "id": 16998,
    "identities": [
      {
        "extern_uid": "2023010622",// 这是学号
        "provider": "thuid"
      }
    ],
    "job_title": "",
    "last_activity_on": "2025-12-11",
    "last_sign_in_at": "2025-12-10T23:07:53.746+08:00",
    "linkedin": "",
    "local_time": "2:46 PM",
    "location": "",
    "locked": false,
    "name": "赵禹晋", // 不一定为真实姓名
    "organization": "",
    "preferred_language": "en",
    "private_profile": false,
    "projects_limit": 100,
    "pronouns": null,
    "public_email": "",
    "state": "active",
    "theme_id": 3,
    "twitter": "",
    "two_factor_enabled": false,
    "username": "zhao-yj23",
    "web_url": "https://git.tsinghua.edu.cn/zhao-yj23",
    "website_url": "",
    "work_information": null
  },
  "message": "Login Successful!",
  "user_id": 16998,
  "username": "zhao-yj23"
}
```


至此，认证流程完成，系统可根据返回的 `id` 或 `email` 匹配本地用户并创建会话。

---

3. 常见问题排查

- **Error: The redirect URI included is not valid**: 请检查代码中的 redirect_uri 是否与 GitLab 后台中填写的完全一致（包括 http/https 和末尾斜杠）。
- **Error: invalid_grant**: 通常是因为 `code` 已过期（有效期很短），或者 `code_verifier` 不匹配。
- **Root Namespace 限制**: 如果链接中包含了 `root_namespace_id`，请确保登录用户属于该指定的 Group，否则授权可能失败。